

Introdução à Computação

operação com strings

Alexandre Rademaker

o tamanho de uma string I

Podemos calcular o tamanho de uma string:

length0.c

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[]) {
4
5     // count up until '\0' (aka NUL or 0)
6     int n = 0;
7     while (argv[1][n] != 0)
8         n++;
9     printf("%i\n", n);
10 }
```

o tamanho de uma string II

E criar uma função para realizar esta tarefa para qualquer string:

length1.c

```
1 #include <stdio.h>
2
3 int string_length(char s[]) {
4     int n = 0;
5     while (s[n] != 0) n++;
6     return n;
7 }
8
9 int main(int argc, char *argv[]) {
10
11     int length = string_length(argv[1]);
12     printf("%i\n", length);
13
14 }
```

o tamanho de uma string III

Mas alguém já implementou isso! função `strlen` da `string.h` !

length2.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(int argc, char *argv[]) {
5
6     printf("%lu\n", strlen(argv[1]) );
7
8 }
```

o tamanho de uma string IV

Veja [string man page](#) ou digite 'man string' (Linux ou MacOS).

imprimindo strings I

Podemos imprimir uma string caractere por caractere.

string0.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(int argc, char *argv[]) {
5
6     printf("Output: ");
7     for (int i = 0; i < strlen(argv[1]); i++)
8         printf("%c", argv[1][i]);
9
10    printf("\n");
11 }
```

imprimindo strings II

Como tornar o código mais eficiente? Não precisamos recalcular a cada interação o tamanho da string!

Veja que 'strlen' agora é usada apenas uma vez na declaração de `n`.

imprimindo strings III

string1.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(int argc, char *argv[]) {
5
6     printf("Output: ");
7     for (int i = 0, n = strlen(argv[1]); i < n; i++)
8         printf("%c", argv[1][i]);
9
10    printf("\n");
11 }
```


transformando em maiúsculas I

Vamos implementar um programa para transformar caracteres em minúsculas para maiúsculas?

transformando em maiúsculas II

uppercase0.c

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main(int argc, char *argv[]) {
5
6     printf("before: %s\n", argv[1]);
7     printf(" after: ");
8     for (int i = 0, n = strlen(argv[1]); i < n; i++) {
9         if (argv[1][i] >= 'a' && argv[1][i] <= 'z')
10             printf("%c", argv[1][i] - 32);
11         else
12             printf("%c", argv[1][i]);
13     }
14     printf("\n");
15 }
```

transformando em maiúsculas III

0	NUL	16	DLE	32	SP	48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	\$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

transformando em maiúsculas IV

Novamente, podemos **reusar** códigos implementados na `ctype.h`.

uppercase1.c

```
1 #include <ctype.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int main(int argc, char *argv[]) {
6
7     printf("before: %s\n", argv[1]);
8     printf(" after: ");
9     for (int i = 0, n = strlen(argv[1]); i < n; i++) {
10         if (islower(argv[1][i]))
11             printf("%c", toupper(argv[1][i]));
12         else
13             printf("%c", argv[1][i]);
14     }
15     printf("\n");
16
17 }
```

transformando em maiúsculas V

E podemos simplificar. Na computação, sempre que possível devemos **reusar**! Mas existem desvantagens em alguns casos. Quais?

uppercase2.c

```
1 #include <ctype.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 int main(int argc, char *argv[]) {
6
7     printf("before: %s\n", argv[1]);
8     printf(" after: ");
9     for (int i = 0, n = strlen(argv[1]); i < n; i++)
10         printf("%c", toupper(argv[1][i]));
11
12     printf("\n");
13 }
```